Synopsis

Air Quality Prediction System using Machine Learning

Problem Id: 15

Student Name:Priyanshi

University Roll number: 2028468

Section:Q

Student Name:Taniya

University Roll number:2028510

Section:Q

Student Name:Ananya Singhal

University Roll number: 2028639

Section:Q

● Problem Description:

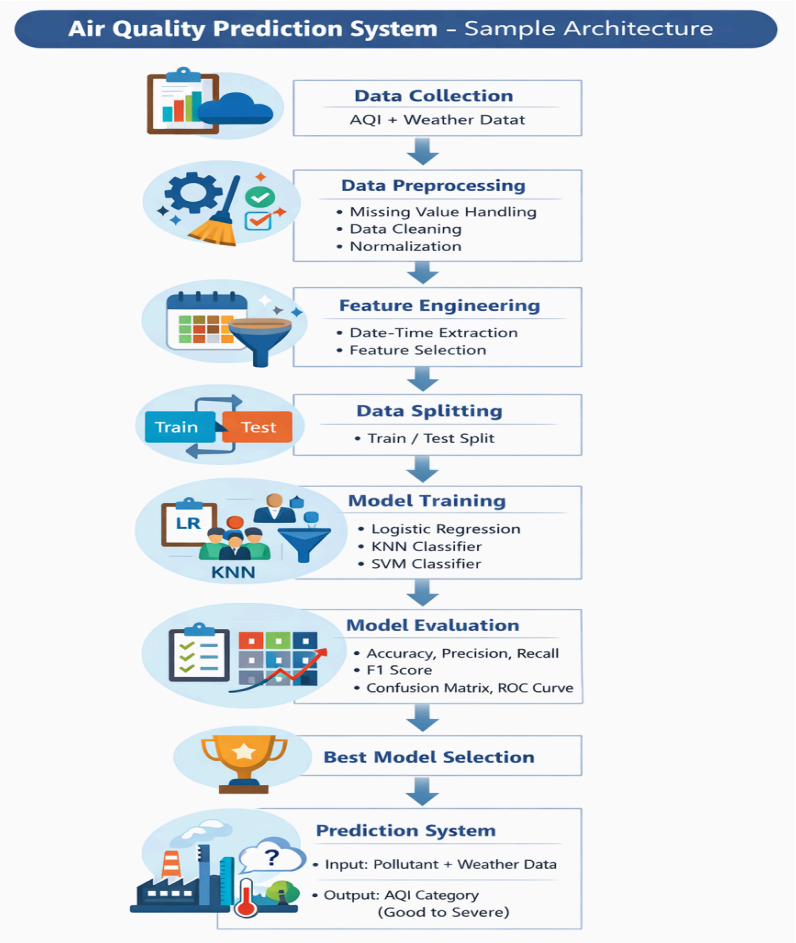
Air pollution is a major environmental and public health issue, especially in rapidly urbanizing regions. The quality of air depends on multiple factors such as concentrations of pollutants like PM2.5, PM10, CO, NO_2, and SO_2, along with meteorological conditions like temperature, humidity, and wind speed. Due to the dynamic and complex nature of these factors, accurately predicting air quality levels is challenging.

This project focuses on developing a machine learning-based system to predict air quality categories (Good to Severe) using historical pollutant and weather data. By analyzing patterns and relationships in the data, the system can classify air quality levels in advance, enabling timely decision-making.

In real life, such a system plays a crucial role in protecting human health by providing early warnings about poor air conditions. It helps governments implement pollution control strategies, supports industries in monitoring emissions, and allows individuals to take preventive measures. Additionally, it contributes to smart city development and environmental sustainability by enabling data-driven planning and policy-making.

● Steps of Implementation:

\



.

1. Data Collection

- Use datasets such as CPCB India AQI dataset or Kaggle air quality datasets.
- Data includes pollutant levels and meteorological parameters.

2. Data Preprocessing

- Handle missing values (mean/median imputation).
- Remove duplicates and outliers.
- Normalize or standardize numerical features.
- Convert date-time into useful features (year, month, day, hour).

3. Feature Selection

- Use correlation analysis and feature importance techniques.
- Select most relevant features like PM2.5, PM10, temperature, humidity.

4. Data Splitting

- Split dataset into training and testing sets (e.g., 80:20 ratio).

5. Model Training

- Train the following classifiers:
    - Logistic Regression
    - K-Nearest Neighbors (KNN)
    - Support Vector Machine (SVM)

6. Model Evaluation

- Evaluate models using classification metrics.
- Compare performance to select the best model.

7. Prediction System

- Input new pollutant values.
- Output predicted air quality category.

Expected Modules/ Libraries to be used :

1. NumPy

Used for numerical computations and handling multi-dimensional arrays efficiently.

2. Pandas

Provides data structures like DataFrames for data cleaning, manipulation, and analysis.

3. Matplotlib

Used for creating basic visualizations such as line plots, bar charts, and graphs.

4. Seaborn

Built on Matplotlib, it helps in creating advanced and more attractive statistical visualizations.

5. Scikit-learn (sklearn)

Provides machine learning algorithms like Logistic Regression, KNN, and SVM along with tools for preprocessing and evaluation.

6. Datetime

Used to handle and extract useful features (day, month, year, hour) from date-time data.

7. Warnings (inbuilt)

Used to suppress unnecessary warning messages during model training and execution.

8. OS (inbuilt)

Helps in interacting with the operating system, such as file handling and path management.

- **Custom Modules**

1. data_preprocessing.py

Contains functions for cleaning data, handling missing values, and normalization.

2. feature_engineering.py

Includes functions for feature selection and transformation like date-time extraction.

3. model_training.py

Handles training of machine learning models (Logistic Regression, KNN, SVM).

4. evaluation.py

Contains functions to calculate metrics like accuracy, precision, recall, and generate confusion matrix.

Expected Evaluation metrics and Plots to be used:

Evaluation Metrics:

- **Accuracy:**
  Measures the overall correctness of the model by calculating the proportion of correctly predicted instances out of total predictions.
- **Precision:**
  Indicates how many of the predicted positive observations are actually correct, helping reduce false alarms.
- **Recall:**
  Measures the model's ability to correctly identify all actual positive cases, which is important for detecting poor air quality conditions.
- **F1-Score:**
  Provides a balance between precision and recall, especially useful when class distribution is imbalanced.
- **Confusion Matrix:**
  Displays the number of correct and incorrect predictions for each class, giving a detailed insight into model performance.
- **ROC Curve and AUC:**
  Evaluates the model's ability to distinguish between different classes and shows the trade-off between true positive rate and false positive rate.

Visualization Plots:

- **Confusion Matrix Heatmap:**
  Helps visualize classification performance in an intuitive matrix form.
- **ROC Curve Plot:**
  Shows how well the model separates different air quality classes.
- **Feature Importance Plot:**
  Highlights the most significant features influencing predictions, improving interpretability.
- **Distribution Plots (Histogram / KDE):**
  Used to understand the distribution of pollutant levels and detect skewness or outliers.

Why These Are Used:

These metrics and plots provide a comprehensive evaluation of the model by measuring accuracy, handling class imbalance, and visually interpreting performance. They help in selecting the best model and ensuring it performs reliably in real-world air quality prediction scenarios.

Required topics from the subject:

1. Strings

Strings are used to handle categorical data such as air quality labels (Good, Moderate, Severe). They also help in processing column names, feature labels, and user inputs in the prediction system.

2. Pointers

Pointers are useful for understanding memory management and efficient data handling at a low level. They provide the foundation for how data is stored and accessed, which is important in optimizing program performance.

3. Files

File handling is used to read input datasets (CSV files) and store outputs such as processed data or model results. It enables persistent storage and retrieval of data for training and evaluation.

4. Functions

Functions help in organizing code into reusable blocks for tasks like preprocessing, training, and evaluation. This improves code readability, modularity, and maintainability.

5. Arrays

Arrays are used to store and manipulate numerical data such as pollutant values and weather parameters. They form the basic structure for performing mathematical operations in machine learning.

Platform Required:

- **Jupyter Notebook / Google Colab:**
  Used for developing and testing the machine learning model interactively with step-by-step execution, visualization, and easy debugging.
- **Python (Anaconda / Python IDLE):**
  Provides the programming environment required to implement data preprocessing, model training, and evaluation using libraries like NumPy, Pandas, and Scikit-learn.
- **Visual Studio Code (VS Code):**
  Used as a code editor for writing, organizing, and managing project files with better extensions and debugging support.

Books and Link Sources:

1. *Hands-On Machine Learning with Scikit-Learn* – Aurélien Géron
2. *Python for Data Analysis* – Wes McKinney
3. Scikit-learn Documentation – https://scikit-learn.org
4. Kaggle Datasets – https://www.kaggle.com
5. CPCB India AQI Data – https://cpcb.nic.in